

1) Univerzální Turingův stroj a nerozhodnutelnost jeho jazyka

def. Turingův stroj (jednopáskový): $M = (Q, \Sigma, \delta, q_0, F)$

Q = mn. stavů (konečná)

Σ = pásková abeceda (konečná), obs. ϵ
nový stav zapiš znak posun hlavy

$\delta: Q \times \Sigma \rightarrow Q \times \Sigma \times \{R, W, L, \perp\}$ přechodová fce

$q_0 \in Q$ počáteční stav $\hookrightarrow$ nedefinovaný přechod

$F \subseteq Q$ přijímající stavy

$\rightarrow$ řídicí jednotka,

obousměrně potenciálně neomezená páska

hlava pro čtení a zápis, která se může pohybovat po pásce

obrázek

pr. 1 slide 32

def. jazyk L je částečně (Turingovsky) rozhodnutelný,
pokud je přijímán nějakým TS

def. jazyk L je (Turingovsky) rozhodnutelný (= rekurzivní)
pokud je přijímán nějakým TS, který se
s každým vstupem zastaví

def. fce $f: \Sigma^* \rightarrow \Sigma^*$ je **turingovsky vyčísitelná**,
pokud $\exists$ TS, který ji počítá

$\downarrow$
 $\rightarrow M(w) \downarrow \Rightarrow f_M(w) \downarrow$ (definovaná fce pro w) $\wedge$
 zastaví $f_M(w)$ = slovo na páse po konci výpočtu
 $M(w) \uparrow \Rightarrow f_M(w) \uparrow$ (nedefinovaná)
 nezastaví

Číslování Turingových strojů

- 1) TS popíšeme řetězcem nad nějakou malou abecedou
- 2) tento řetězec převedu do binární abecedy
- 3) řetězci přiřadíme číslo podle pořadí v shortlex usp.

$\downarrow$
 $\text{index}(w) = i \rightarrow 1.w = (i+1)_B$
 $\hookrightarrow$ konkatence 1 a w

Gödelovo číslo

předpoklady:

- jediný přijímající stav
- binární vstupní abeceda $\Sigma_{in} = \{0, 1\}$
(neomezená pracovní abeceda)

$\hookrightarrow$ každý TS lze upravit tak, aby splňoval oba předpoklady

zakódování přechodové fce

- řetězec w_M nad abecedou $\Gamma = \{0, 1, L, N, R, /, \#, i\}$
 $\rightarrow$ každý znak w_M zapíše pomocí 3 bitů

$\downarrow$

$\langle M \rangle$ = binární řetězec reprezentující M

- očíslované stavy: $Q = \{q_0, q_1, \dots, q_r\}$, $r \geq 1$

q_0 = počáteční, q_1 = přijímající

očíslované symboly : $\Sigma = \{x_0, x_1, \dots, x_s\}$, $s \geq 2$

$$x_0 = 0, x_1 = 1, x_2 = 2$$

$\delta(q_i, x_j) = (q_k, x_l, z)$, $z \in \{L, N, R\}$

$\downarrow$
 $\rightarrow (i)_B | (j)_B | (k)_B | (l)_B | z \rightarrow \in \Gamma^*$

instrukce $c_1, \dots, c_n \rightarrow c_1 \# \dots \# c_n$

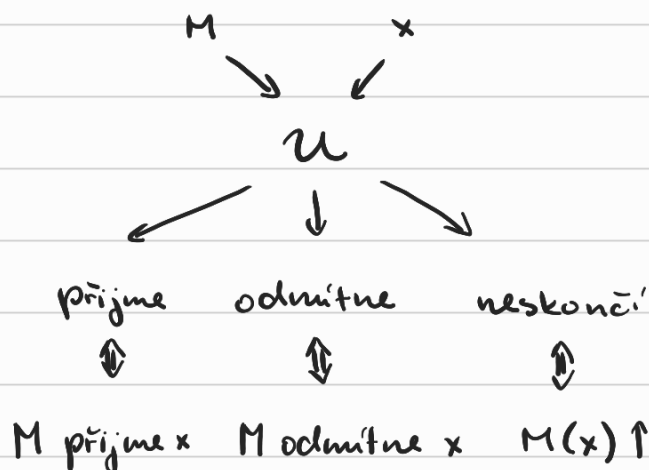
$\langle M \rangle$ = kód Turingova stroje M

$\langle M, x \rangle$ = kód dvojice tvořené TS M a řetězcem x

Univerzální TS U

• vstup : $\langle M, x \rangle$

• simuluje práci TS M nad vstupem x



def. univerzální jazyk $L_u = \{ \langle M, x \rangle \mid x \in L(M) \}$

= jazyk univerzálního Turingova stroje

→ formalizace problému přijetí vstupu :

instance : TS M a vstupní řetězec x

otázka : přijme M vstup x ?

3-páskový UTS $\mathcal{U}$

- 1) vstupní páska $\rightarrow$ vstup $\langle M, x \rangle$
- 2) obsah pracovní pásky M
 $\rightarrow$ symbol x_j zapsán jako $(j)_B$
- 3) stav M
 $\rightarrow$ stav q_i zapsán jako $(i)_B$

inicializace:

- 1) $\langle M \rangle$ není syntakticky správný kód TS $\rightarrow$ odmítnutí
- 2) určení # bitů pro zápis znaku (velikost bloku = b)
- 3) přepis vstupu z 1. pásky na 2.
(b bitové kódování znaků, oddělené 1)
- 4) zápsání 0 na 3. pásku (stav q_0)
- 5) na 4. pásku návrat hlavy na začátek

simulace kroku M

- 1) hledej v $\langle M \rangle$ instrukci pro displej (q_i, x_j)
 $\left\{ \begin{array}{l} \text{nenalezena} \rightarrow \text{konec simulace} \\ \delta(q_i, x_j) = (q_k, x_e, z) \end{array} \right.$
- 2) 3. páska $\rightarrow$ přepsat na $(k)_B$
- 3) 2. páska $\rightarrow$ přepsat blok pod hlavou na $(l)_B$
 $\rightarrow$ posunutí o blok vlevo/vpravo / na začátek bloku
- 4) hlava mimo použitou část pásky $\rightarrow$ nový blok $0^{b-2} 10$ ($x_2 = \lambda$)
- 5) návrat hlav do předpokládaných pozic

zakočení:

3. páska číslo 1 ? přijmi : odmítní
pokud M počítá $fci \rightarrow$ přepiš 2. pásku do Σ^*

V: $L_u = \{ \langle M, x \rangle \mid x \in L(M) \}$ je částečně rozhodnutelný, ale není rozhodnutelný

ok.

částečná rozhodnutelnost plyne z existence UTS a nerozhodnutelnost ukažeme diagonalizací:

- 1) L_u reprezentujeme jako matici A
- 2) jazyk daný doplňkem diagonály A není částečně rozhodnutelný
- 3) z toho dovodíme, že L_u není rozhodnutelný

reprezentace L_u nekonečnou maticí A :

Gödelova čísla $\langle M_i \rangle$	indexy bin. řetězců (w_i)						
	0	1	2	...	i	...	j
0	0	1	0		0		1
1	1	1	0		1		0
2	1	1	0		0		1
...							
i					1		0
...							

$\downarrow$
 $w_i \in L(M_i)$

$\textcircled{1} \quad \textcircled{0} \rightarrow w_j \notin L(M_i)$

$\forall$ TS má nekonečně mnoho Gödelových čísel

$\rightarrow \forall$ TS odpovídá nekonečně mnoho řádků A

$\rightarrow \forall$ částečně rozhodnutelnému $L \dashv$

bud' $\text{DIAG} = \{ \langle M \rangle \mid \langle M \rangle \in L(M) \}$ diagonální jazyk
 = doplněk diagonály matice A

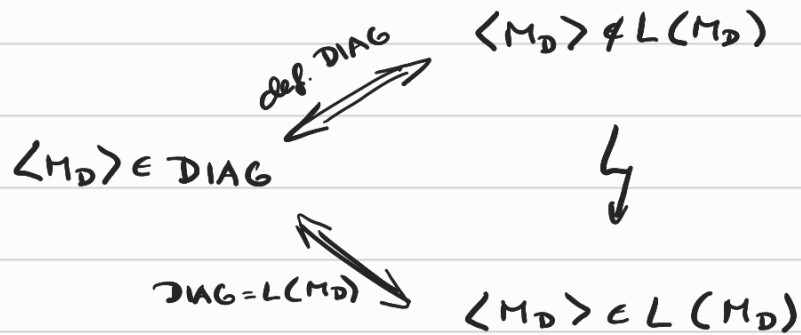
$\rightarrow$ nemá svůj řádek v A (od $\forall$ se liší v diagonale)

$\rightarrow$ není částečně rozhodnutelný

V: DIAG není částečně rozhodnutelný

dk. sporem:

$\exists$ TS M_D t.ž. $DIAG = L(M_D)$



$\downarrow$

$$\langle M \rangle \in DIAG \stackrel{\text{def. DIAG}}{\Leftrightarrow} \langle M \rangle \notin L(M) \stackrel{\text{def. } L_u}{\Leftrightarrow} \langle M, \langle M \rangle \rangle \notin L_u$$

tedy

M_u lze použít k rozhodování DIAG

$\rightarrow$ $\nexists$ se nerozhodnutelností DIAG

□

2) RAM a ekvivalence s TS

def. Random Access Machine

→ řídicí jednotka (procesor, CPU) a neomezená paměť

obrázek (př. 2 slide 4)

- paměť RAMu rozdělena do registrů r_i , $i \in \mathbb{N}$
- v $\forall r_i$ může být libovolné $\mathbb{N}_0$ (0 na začátku)
- $[r_i]$ = obsah registru
- $[[r_i]] = [r_{[r_i]}]$ → nepřímá adresace
- program pro RAM = posloupnost instrukcí (konečná)

- abeceda $\Sigma = \{\sigma_1, \sigma_2, \dots, \sigma_k\}$
- RAM čte slovo $\sigma_{i_1}\sigma_{i_2}\dots\sigma_{i_n}$ jako posl. čísel $i_1, \dots, i_n, 0$
- přijme $w \Leftrightarrow$ skončí a 1. číslo zapsané na výstup je 1
- odmítne $w \Leftrightarrow$ skončí a nezapiše nic nebo 1. zapsané $\neq 1$

TS $\rightarrow$ RAM

V: Ke každému Turingovu stroji M existuje ekvivalentní RAM R

! předpokládám, že páska M je neomezená pouze doprava

def. konfigurace zachycuje stav výpočtu TS

→ stav řídicí jednotky, slovo na páse,
pozici hlavy na páse (v rámci slova)

• ekvivalentní RAM musí ve své paměti reprezentovat konfiguraci TS

→ očísloji stavy $\in Q \rightarrow q_0, \dots, q_r$ ($q_0 = \text{počáteční}$)

→ očísloji symbols $\in \Sigma \rightarrow \sigma_0, \dots, \sigma_{s-1}$ ($\sigma_0 = \epsilon$)

(0 na konci vstupu R odpovídá prázdnému políčku)

→ pole $T = \text{obsah pásky}$ ($T[0] = 1. \text{znak vstupu}$)

→ proměnná $h = \text{poloha hlavy}$

→ proměnná $q = \text{stav}$

Program R:

1. načti vstup do $T \rightarrow \text{poslední znak} = 0$

2. $q = 0, h = 0$

3. Dokud je $\delta(q, T[h])$ definována:

odsimuluj krok určený přechodovou fci

→ aktualizace $T[h], q, h$

(dane' velkým if-else)

4. Pokud nás zajímá přijetí:

$q \in F ? \text{ print } 1 : \text{ print } 0$

5. Pokud nás zajímá obsah pásky:

vypiš na výstup obsah pásky

RAM $\rightarrow$ TS

V: Ke každému RAMu existuje ekvivalentní TS M.

4 pásky:

1) vstupní (jen pro čtení)

→ posloupnost čísel, která má dostat R na vstup

(zako'dována bina'rně, oddělena #)

2) výstupní (jen pro zápis)

→ čísla, která R zapisuje na výstup

(zakódována binárně, oddělena #)

3) paměť RAM

$r_{i_1}, \dots, r_{i_m}$ = aktuálně využívané registry ($i_1 < \dots < i_m$)

→ páska:

$(i_1)_B \mid ([r_{i_1}])_B \# \dots \# (i_m)_B \mid ([r_{i_m}])_B$

$\uparrow$
binární zápis

$\hookrightarrow$ oddělovací znaky

4) pomocná páska

→ výpočty (součet, rozdíl, nepřímá adresace,

posun části paměťové pásky apod.)

- číslo prováděné instrukce je uloženo ve stavu

- instrukce R ~ řada instrukcí M

- nalezení registrů s operandy instrukce

- provedení aritmetických operací

- výpočet adres nepřímé adresace

- úprava paměti podle výsledku instrukce

- přidání registru, přepis hodnoty v registru

- může být nutné posunout část pásky

- simulace končí, když už nenásleduje další instrukce

- přijetí $\Leftrightarrow$ na výstupní pásce je pouze číslo 1

(lze si pamatovat i ve stavu)

3) Vlastnosti (částečně) rozhodnutelných jazyků

def. jazyk $L \subseteq \Sigma^*$ je

- částečně rozhodnutelný, je-li přijímán nějakým Turingovým strojem M ($L = L(M)$)
(= rekurzivně spočetný jazyk)
- rozhodnutelný, je-li přijímán nějakým TS M a výpočet s každým vstupem se zastaví
 $(L = L(M) \wedge (\forall x \in \Sigma^*) M(x) \downarrow)$
(= rekurzivní jazyk)

částečně rozhodnutelných jazyků nad Σ je spočetně mnoho

$\rightarrow \forall$ přijímá nějaký TS $\rightarrow$ ten má Gödelovo číslo

$\rightarrow$ mapování na $\mathbb{N}$

def. $\bar{L} = \Sigma^* \setminus L$ doplněk jazyka L nad Σ

$L_1 \cdot L_2 = \{w_1 w_2 \mid w_1 \in L_1 \wedge w_2 \in L_2\}$ konkatenace jazyků

$L^* = \{w \mid (\exists k \in \mathbb{N}) (\exists w_1, \dots, w_k \in L) [w = w_1 w_2 \dots w_k]\}$

Kleeneho uzavření

V: Pro (částečně) rozhodnutelné L_1 a L_2 platí:

$L_1 \cup L_2$, $L_1 \cap L_2$, $L_1 \cdot L_2$ a L_1^* jsou (částečně) rozhodnutelné

V: (Postova)

L je rozhodnutelný $\Leftrightarrow L$ i $\bar{L}$ jsou částečně rozhodnutelné

dk. $\Rightarrow$

$\exists$ TS M rozhodující L

sestrojíme M' t.č. $M'(x)$ přijme $\Leftrightarrow M(x)$ odmítne

$\rightarrow$ TS přijímající $\bar{L}$

$\rightarrow \bar{L}$ je rozhodnutelný

$\Leftarrow$ TS M_1 přijímající L } ale nemusí se zastavit
 TS M_2 přijímající $\bar{L}$

↓

chci $M : L(M) = L \wedge (\forall x) Mx \downarrow$

↓ paralelní výpočet M_1 a M_2

1) přijme $M_1 \rightarrow x \in L(M)$
 2) přijme $M_2 \rightarrow x \notin L(M)$
 → 1 se vždy zastaví

□

Diskolek :

- 1) rozhodnutelné jazyky jsou uzavřené na doplněk
- 2) částečně rozhodnutelné jazyky nejsou uzavřené na doplněk

př. 2 $DIAG$ je nerozhodnutelný a $\overline{DIAG}$ je část. rozhodnutelný
 $\overline{L_u}$ není částečně rozhodnutelný (L_u není rozhodnutelný)

def. Pro částečnou fci $f: \Sigma^* \rightarrow \Sigma^*$:

doména $\text{dom } f = \{x \in \Sigma^* \mid f(x) \downarrow\}$

= mn. vstupů, pro které je fce definována

totální fce = fce definována pro $\forall x \in \Sigma^*$

obor hodnot $\text{rng } f = \{y \in \Sigma^* \mid (\exists x \in \Sigma^*) [f(x) \downarrow = y]\}$

= mn. možných hodnot fce

def. částečná fce $f: \Sigma^* \rightarrow \Sigma^*$ je **algoritmiicky vyčíslitelná**,

pokud $\exists$ TS M , který ji počítá (= částečně rekurzivní)

$\forall x : f(x) \uparrow \Rightarrow M(x) \uparrow$

$f(x) \downarrow = y \Rightarrow M(x) \downarrow$ a na výstupu M je y

• vyčíslitelných fci je spočetně mnoho (stejně jako TS)

V: $L \subseteq \Sigma^*$ je rozhodnutelný $\Leftrightarrow \chi_L$ je algoritmiicky vyčíslitelná

$$\bullet \chi_L(x) = \begin{cases} 1 & x \in L \\ 0 & x \notin L \end{cases} \quad (\text{charakteristická fce } L)$$

olk. $\Rightarrow$

TS M přijímá L a vždy skončí

M' : M přijme $\rightarrow 1$

M odmítne $\rightarrow 0$

$\Leftarrow$

TS M , který počítá χ_L

M' : přijme, když M vrátí 1

odmítne jinak

V: L je částečně rozhodnutelný $\Leftrightarrow \exists$ TS M : $L = \{x \in \Sigma^* \mid M(x) \downarrow\}$

$\rightarrow$ místo přijetí otevíráme zastavení

olk. $\Rightarrow$

$\exists$ TS M' : $L = L(M')$

M : M' přijme $\rightarrow$ přijmu

M' odmítne $\rightarrow$ přejdu do nekonečného cyklu

$\Leftarrow$

$\exists$ TS M :

M' : když M zastaví, tak přijmu

V: L je částečně rozhodnutelný $\Leftrightarrow \exists f$ algoritmiicky vyčíslitelná
t. j. $L = \text{dom } f = \{x \in \Sigma^* \mid f(x) \downarrow\}$

olk. plyne z $\exists$ TS, které se na vstupu zastaví

V: L je částečně rozhodnutelný $\Leftrightarrow \exists$ rozhodnutelný jazyk B
 t.j. $L = \{x \in \Sigma^* \mid (\exists y \in \Sigma^*) [\langle x, y \rangle \in B]\}$

dk $\Rightarrow$

$\exists$ TS M přijímající L $\rightarrow$ jinak by M nepřijal

$$\rightarrow L = \{x \mid (\exists n \in \mathbb{N}) [M(x) \text{ přijme do } n \text{ kroků}]\}$$

rozhodnutelná podmínka
(stačí simulovat $M(x)$ po n krocích)

def. $B = \{\langle x, \langle n \rangle \rangle \mid M(x) \text{ přijme do } n \text{ kroků}\}$

$\rightarrow$ rozhodnutelný a splňuje podmínku

$\Leftarrow$

$\exists$ TS M , který přijímá B a vždy skončí

postupně zkusím $\forall y \in \Sigma^*$ v shortlex uspořádání

$\hookrightarrow M$ přijme $\rightarrow$ přijmu

M nepřijme $\rightarrow$ zkusím další

(nemusí skončit $\rightarrow$ částečná rozhodnutelnost)

Důsledek:

Je-li B částečně rozhodnutelný, pak

$$A = \{x \in \Sigma^* \mid (\exists y \in \Sigma^*) [\langle x, y \rangle \in B]\}$$

je také částečně rozhodnutelný.

dk.

$\exists$ rozhodnutelná C :

$$B = \{\langle x, y \rangle \in \Sigma^* \mid (\exists z \in \Sigma^*) [\langle x, y, z \rangle \in C]\}$$

$$\text{platí } A = \{x \in \Sigma^* \mid (\exists \langle y, z \rangle \in \Sigma^*) [\langle x, y, z \rangle \in C]\}$$

$\rightarrow$ č.r. platí z věty

□

def. **enumerátor** pro jazyk L je TS E t.ž.

- ignoruje svůj vstup
- vypisuje řetězce $w \in L$ na vyhrazenou výstupní pásku
- $\forall w \in L$ je někdy vypsan
- pro nekonečný L se E nikdy nezastaví

V: L je částečně rozhodnutelný $\Leftrightarrow$ pro něj existuje enumerátor E

dk. $\Rightarrow$

$\exists$ rozhodnutelný $B : L = \{x \mid \exists y : \langle x, y \rangle \in B\}$

enumerátor :

$\forall \langle x, y \rangle \in \Sigma^*$ v shortlexu :

$\langle x, y \rangle \in B$? vypiš x

(! vypisování v neznámém pořadí)

$\Leftarrow$

TS simuluje enumerátor :

vypišu $x \rightarrow$ přijmu x

pro $x \notin L$ nekonečný TS

V: L je rozhodnutelný $\Leftrightarrow \exists E$, který jeho prvky vypíše
v lexikografickém pořadí

dk. $\Rightarrow$

$\forall x \in \Sigma^*$ v lexikografickém pořadí :

$x \in L$? vypišu

$\hookrightarrow$ lze ověřit díky rozhodnutelnosti

$\Leftarrow$

1) L je konečný $\rightarrow \forall$ konečný L je rozhodnutelný

2) L je nekonečný

E vypíše $x \rightarrow$ přijmu

E vypíše $y > x \rightarrow$ odmitnu

Důsledek:

nekonečný

L je částečně rozhodnutelný $\Leftrightarrow L = \text{rng } f$ (včetně f) ^{totalní} ^{def pro $\forall x$}

dk. $\Rightarrow$

E enumerator L

pro jednoduchost parametry f typu $\mathbb{N}$

pro $i \in \mathbb{N}$ def. $f(i) = (i+1)$ n řetězec vypsaný E

$\Leftarrow$

$\forall y \in \Sigma^*$ v lex pořadí:

$f(y)$ vypíšu na výstup

$\hookrightarrow$ totalní $\rightarrow$ vždy získám hodnotu

Důsledek

nekonečný L je rozhodnutelný $\Leftrightarrow L = \text{rng } f$

f = rostoucí totalní fce

dk. $\Rightarrow$

E vypisuje prvky v lex pořadí

$f(i) \stackrel{\text{def}}{=} (i+1)$ n řetězec vypsaný E

$\rightarrow i < j \rightarrow f(i) < f(j)$

$\Leftarrow$

postupně vypíšu $f(y)$ $\forall y \in \Sigma^*$

f je rostoucí $\rightarrow$ lex pořadí

4) Základní třídy složitosti, dikaz $\text{NTIME}(f(n)) \subseteq \text{SPACE}(f(n))$

def. Necht' M je deterministický TS a $f: \mathbb{N} \rightarrow \mathbb{N}$ je fce,
která je definována pro každý vstup

M pracuje v čase $f(n)$, pokud výpočet M nad libovolným
vstupem x délky $|x|=n$ skončí po provedení nejvýše
 $f(n)$ kroků

M pracuje v prostoru $f(n)$, pokud výpočet M nad libovolným
vstupem x délky $|x|=n$ skončí a využije nejvýš $f(n)$
buněk pracovní pásky

def. Necht' $f: \mathbb{N} \rightarrow \mathbb{N}$ je fce, pak definujeme třídy

$\text{TIME}(f(n))$ = třída jazyků přijímaných TS, které
pracují v čase $O(f(n))$

$\text{SPACE}(f(n))$ = třída jazyků přijímaných TS, které
pracují v prostoru $O(f(n))$

$\forall f: \mathbb{N} \rightarrow \mathbb{N} : \text{TIME}(f(n)) \subseteq \text{SPACE}(f(n))$

$\rightarrow$ čas je vždy alespoň tolik co prostor

def. Necht' M je nedeterministický TS a $f: \mathbb{N} \rightarrow \mathbb{N}$ je fce

M pracuje v čase $f(n)$, pokud $\exists$ výpočet M nad libovolným
vstupem x délky $|x|=n$ skončí po provedení nejvýše
 $f(n)$ kroků

M pracuje v prostoru $f(n)$, pokud $\exists$ výpočet M nad libovolným
vstupem x délky $|x|=n$ skončí a využije nejvýš $f(n)$
buněk pracovní pásky

def. Necht' $f: \mathbb{N} \rightarrow \mathbb{N}$ je fce, pak definujeme

$\text{NTIME}(f(n))$ = třída jazyků přijímaných NTS, které pracují v čase $O(f(n))$

$\text{NSPACE}(f(n))$ = třída jazyků přijímaných NTS, které pracují v prostoru $O(f(n))$

T: Pro $\forall f: \mathbb{N} \rightarrow \mathbb{N}$ platí:

$$\text{TIME}(f(n)) \subseteq \text{NTIME}(f(n))$$

$$\text{SPACE}(f(n)) \subseteq \text{NSPACE}(f(n))$$

V: Pro $\forall f: \mathbb{N} \rightarrow \mathbb{N}$ platí:

$$\text{NTIME}(f(n)) \subseteq \text{SPACE}(f(n))$$

dk.

- Necht' L je přijíman NTS M v čase $g(n) = O(f(n))$
- předp. $M = (Q, \Sigma, \delta, q_0, F)$
- $r = \max_{\substack{q \in Q \\ a \in \Sigma}} |\delta(q, a)| \leftarrow \text{max. větvicí faktor}$
 $\rightarrow r \leq |Q| \cdot |\Sigma| \cdot 3$
- výpočet M se vstupem x lze popsat řetězcem $y \in \{1, \dots, r\}^{g(|x|)}$
- y_i = větev zvolená v kroku i , $i=1, \dots, g(|x|)$
- popišu TS M' , který rozhoduje L v prostoru $O(f(n))$
 $\hookrightarrow$! deterministický

M' :

$k \leftarrow 1$

opakuji než $\forall$ simulace $M(x)$ odmítne:

$\forall y \in \{1, \dots, r\}^k \leftarrow g(|x|)$ nemusí být vyčíslitelná v prostoru $O(g(n))$
 $\rightarrow$ iterativně zvyšuji

simuluj $M(x)$ s větvením podle y

pokud větev výpočtu přijme $\rightarrow$ přijme

$k \leftarrow k+1$

odmítni

M pracuje v čase $g(n) = O(f(n))$

H výpočet $M(x)$ skončí do $g(n)$ kroků

$M'(x)$ skončí výpočet s $k \leq g(|x|)$

uložení $y \rightarrow$ prostor $O(g(n)) = O(f(n))$

simulace $M(x)$ podle $y \rightarrow$ prostor $O(g(n)) = O(f(n))$

↓

M' pracuje v prostoru $O(f(n))$

$L = L(M')$

$\left\{ \begin{array}{l} M \text{ přijme} \rightarrow M' \text{ přijme} \\ M \text{ odmítne} \rightarrow M' \text{ odmítne} \end{array} \right.$

□

5) Základní třídy složitosti, důkaz věty o vztahu prostoru a času

- viz. 4 -

V: (Vztah prostoru a času)

$\forall f \in f(n) \geq \log_2 n$ a $\forall$ jazyk L platí:

$$L \in \text{NSPACE}(f(n)) \Rightarrow (\exists c_L \in \mathbb{N}) [L \in \text{TIME}(2^{c_L f(n)})]$$

dk.

$\exists$ NTS M přijímající L v prostoru $O(f(n))$

pro vstup x def graf konfigurací $G_{M,x}$

$x \in L \Leftrightarrow G_{M,x}$ obsahuje cestu z počáteční konfigurace
do nějaké přijímající konfigurace

Konfigurace :

- stav $|Q|$
 - poloha hlavy na vstupní páse n
 - poloha hlavy na pracovní páse $f(n)$
 - slovo na pracovní páse ($|w| \leq f(n)$) $|\Sigma|^{f(n)}$
- # konfigurací $\leq |Q| \cdot n \cdot f(n) \cdot |\Sigma|^{f(n)}$

$G_{M,x}$ = orientovaný graf popisující přechody mezi konfiguracemi

L: $f(n) \geq \log_2 n$ je $|V| \leq 2^{c_M f(n)}$, c_M = konstanta

$\forall$ vrcholy $G_{M,x}$, TS M pracující v prostoru $f(n)$

$$\begin{aligned} |V| &\leq |Q| \cdot n \cdot f(n) \cdot |\Sigma|^{f(n)} = 2^{\log_2 |Q|} \cdot 2^{\log_2 n} \cdot 2^{\log_2 f(n)} \cdot 2^{f(n) \cdot \log_2 |\Sigma|} = \\ &= 2^{\log_2 |Q| + \log_2 n + \log_2 f(n) + f(n) \cdot \log_2 |\Sigma|} \leq 2^{f(n) \cdot (\log_2 |Q| + 1 + 1 + \log_2 |\Sigma|)} \\ &\quad \text{konstanta daná } M \\ &\quad = c_M \end{aligned}$$

$\hookrightarrow |E| \leq |V|^2 \leq 2^{2c_M f(n)}$

□

→ prohledávání do hloubky na grafu konfigurací:

- 1) naleznou cestu → přijmu
- 2) nenaleznou cestu → odmítnu

↓
v polynomiálním čase vůči velikosti grafu

$$\rightarrow O(|V|^k) = O(2^{k \cdot \log_2 |V|}) = O(2^{c \cdot f(n)})$$

konstanty $k \geq 1$ a $c \geq k \cdot \log_2 |V|$

Důsledek:

pro $f(n) \geq \log_2 n$, $g(n)$ t.j. $f(n) = o(g(n))$, pak:

$$\text{NSPACE}(f(n)) \subseteq \text{TIME}(2^{g(n)})$$

6) 2 definice NP a jejich ekvivalence

def. algoritmus V je **verifikátor** pro jazyk A , když

$$A = \{x \mid (\exists y) [V(x, y) \text{ přijme}] \}$$

↓

$$\hookrightarrow y = \text{certifikát } x$$

polynomiální verifikátor = pracuje v čase $O(|x|^k)$, $k \in \mathbb{N}$

→ stačí uvažovat y s délkou polynomiální v $|x|$

def. NP = třída jazyků, které mají polynomiální verifikátory

↓

třída jazyků přijímaných NTS v polynomiálním čase

→ úlohy, u kterých jsme schopni v polynomiálním čase ověřit, zda je daný řetězec polynomiální délky jejich řešením

V: (alternativní definice třídy NP)

$$NP = \bigcup_{k \in \mathbb{N}} \text{NTIME}(n^k)$$

dk. $\Rightarrow$

$$NP \subseteq \bigcup_{k \in \mathbb{N}} \text{NTIME}(n^k)$$

• máme polynomiální verifikátor V pro jazyk L

→ popíšeme NTS M přijímající L v polynomiálním čase

M :

1. simuluj verifikátor V

2. nedeterministicky zvol znaky y , když je V potřebuje číst

3. V přijme $\rightarrow$ přijmeme

odmítne $\rightarrow$ odmítneme

$$x \in L \Leftrightarrow \exists y (|y| \leq p(|x|) \wedge V(x, y) \text{ přijme})$$

$$\Leftrightarrow \text{nějaký výpočet } M(x) \text{ přijme} \Leftrightarrow x \in L(M)$$

⇐

$$\bigcup_{k \in \mathbb{N}} \text{NTIME}(n^k) \subseteq \text{NP}$$

• NTS M pracující v polynomiálním čase $p(n)$

$$r = \max_{\substack{q \in Q \\ a \in \Sigma}} |\delta(q, a)| \leftarrow \text{max. větvící faktor}$$

$$\rightarrow r \leq |Q| \cdot |\Sigma| \cdot 3$$

↳ nezávisí na vstupu $\rightarrow$ konstanta

$$y \in \{1, \dots, r\}^{p(|x|)} \rightarrow \text{řetězec popisující výpočet } M(x)$$

y_i = větev zvolená v kroku i

↳ y = certifikát toho, že M přijímá x

verifikátor V :

• simuluje $M(x)$ podle y

• větev $M(x)$ přijme $\rightarrow$ přijme

jinak $\rightarrow$ odmítne

$x \in L \Leftrightarrow$ nějaký výpočet $M(x)$ přijme

$\Leftrightarrow \exists y [|y| \leq p(|x|) \wedge \text{simulace } M(x) \text{ podle } y \text{ přijme}]$

$\Leftrightarrow \exists y [|y| \leq p(|x|) \wedge V(x, y) \text{ přijme}]$

12) Třída co-NP a co-NP úplnost

def. $\text{co-NP} = \{A \mid \bar{A} \in \text{NP}\}$

TAUT

instance: výroková formule φ
otázka: je φ tautologie
↓

$\text{TAUT} = \{\langle \varphi \rangle \mid \varphi \text{ je tautologie}\}$

$\overline{\text{TAUT}} = \{\langle \varphi \rangle \mid \varphi \text{ není tautologie}\}$

↓ $\hookrightarrow \Leftrightarrow \varphi(a) = 0$ pro nějaké ohodnocení a
 $\exists$ poly verifikátor $V(\varphi, a)$, který ověří, zda $\varphi(a) = 0$
 $\rightarrow \overline{\text{TAUT}}$ patří do NP
 $\rightarrow \text{TAUT}$ patří do co-NP

$A \in \text{co-NP} \Leftrightarrow \exists$ poly verifikátor $V : A = \{x \mid (\forall y \in \{0,1\}^*) [V(x,y) \text{ přijme}]\}$

✓ $\vee$ NP může odmítnout
 $\vee$ NP je $\exists$ proto je NP verifikovat

$P \subseteq \text{NP} \cap \text{co-NP}$

def. jazyk B je co-NP-těžký, pokud $\forall A \in \text{co-NP} : A \leq_m^P B$

co-NP-úplný, pokud je co-NP-těžký $\wedge B \in \text{co-NP}$

V: A je co-NP-úplný $\Leftrightarrow \bar{A}$ je NP-úplný

dk.

protože $\forall A, B : A \leq_m^P B \Leftrightarrow \bar{A} \leq_m^P \bar{B}$

SAT je NP-úplný $\rightarrow \overline{\text{SAT}}$ je co-NP-úplný

V: Pokud nějaký NP-úplný problém A patří do co-NP ,
pak $\text{NP} = \text{co-NP}$

dk.

$$A \in \text{co-NP} \rightarrow \bar{A} \in \text{NP}$$

$$B \in \text{NP} \rightarrow B \leq_m^P A$$

$$\bar{B} \leq_m^P \bar{A} \rightarrow \bar{B} \in \text{NP} \rightarrow B \in \text{co-NP}$$

$$\rightarrow \text{NP} \subseteq \text{co-NP}$$

symetricky $\text{co-NP} \subseteq \text{NP}$

11) Třída #P a #P-úplnost, důkaz těžkosti počítání cyklů v grafu

#CYCLE

instance: orientovaný graf $G = (V, E)$

otázka: počet cyklů G

V: Pokud lze #CYCLE vyčíslit v polynomiálním čase, pak $P = NP$
→ šlo by využít k řešení Hamiltonovské kružnice

def. fce $f: \Sigma^* \rightarrow \mathbb{N}$ patří do třídy $\#P$, pokud

$\exists$ polynomialní verifikátor V a polynom $p(n)$ t.j.

$$\forall x \in \Sigma^*: f(x) = |\{y \in \{0,1\}^{p(|x|)} \mid V(x,y) \text{ přijme}\}|$$

alternativně pomocí NTS M :

$$f(x) = \# \text{ přijmavých výpočtů } M(x)$$

$NP = \exists$ poly certifikát pro daný vstup?

$\#P =$ kolik certifikátů pro daný vstup existuje?

$NP \rightarrow SAT \rightarrow \exists$ model?

$\#P \rightarrow \#SAT \rightarrow$ kolik $\exists$ modelů?

· počítání certifikátů může být těžší než hledání jednoho

9) FPT, kernel, kernelizace VP

def. parametrizovaný problém = jazyk $L \subseteq \Sigma^* \times \mathbb{N}$

Σ = konečná abeceda

$\langle I, k \rangle \in \Sigma^* \times \mathbb{N}$, k = parametr instance

$| \langle I, k \rangle | = |I| + k \rightarrow$ velikost instance

def. parametrizovaný problém $L \subseteq \Sigma^* \times \mathbb{N}$ je řešitelný s pevným parametrem, právě když je rozhodnutelný alg A , který pracuje v čase $O(f(k) \cdot |I|^c)$

f = algoritmicke vyčísitelná fee

c = konstanta

$\rightarrow A$ = parametrizovaný algoritmus pro L

def. FPT = třída problémů řešitelných s pevným parametrem

def. $O^*(f(k))$ = třída funkcí $O(f(k) \cdot n^c)$ pro libovolnou konstantu c

$\rightarrow$ zachycuje závislost na parametru k ,

pomáháme polynomiální faktor

kernel:

$\langle I, k \rangle$



kernelizační alg.

(polynomiální čas)



$\langle I', k' \rangle$ ($|I'| + k' \leq g(k)$)

$\rightarrow$ vyčísitelná

$(\langle I, k \rangle \in L \Leftrightarrow \langle I', k' \rangle \in L)$ $\rightarrow$ velikost kernelu

$\hookrightarrow$ kernel

obvykle $k' \leq k$

kernelizace VP

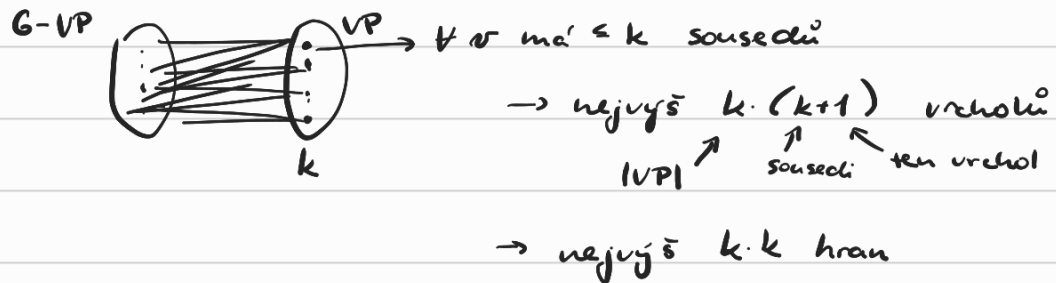
VP1: pokud $\exists$ izolovaný uzel, tak ho smaž

$\rightarrow \langle G-v, k \rangle$

VP2: pokud $\exists$ uzel v stupně $\geq k+1$, v jde rovnou do VP

$\rightarrow \langle G-v, k-1 \rangle$

L: nelze aplikovat VP1 ani VP2 $\rightarrow |V| \leq k^2+k, |E| \leq k^2$



VP3: VP1 a VP2 nelze a zároveň:

$k < 0$ nebo $|V| > k^2+k$ nebo $|E| > k^2$

pak G nemá VP velikosti nejvýš k

kernelizační alg.:

$O(k)$ $\left\{ \begin{array}{l} \text{opakově dokud se nemění hodnota } k \text{ nebo } k < 0 \\ \text{odstranit izolované vrcholy: } G \leftarrow G-v \\ \text{vyřeš } v: \deg(v) > k \quad G \leftarrow G-v, k \leftarrow k-1 \end{array} \right\} O(|G|)$

if $k < 0$ nebo $|V| > k^2+k$ nebo $|E| > k^2$ then

return neg. instanci (př. graf s 1 hranou a $k=0$)

return $\langle G, k \rangle$

$\hookrightarrow$ kernel s $O(k^2)$ vrcholy a $O(k^2)$ hranami

L: rozhodnutelný parametrizovatelný problém L:

$L \in \text{FPT} \Leftrightarrow$ má kernel

